

Name :- Nimit jain

Reg.No :- 25BAI10840

Branch :- CSE(AIML)

Faculty :-Dr J. Sharmila Joseph

VIT BHOPAL
PROJECT REPORT

Title :- Banking-Management-System

Introduction:-

This project, the Banking Management System is a console-based application developed using the Java programming language. It is designed to simulate a fundamental core banking and account management system for a small branch, credit union, or financial unit. The core objective is to demonstrate the mastery of introductory and intermediate Computer Science principles, including modular programming, object-oriented design, fundamental data structures (Collections and Maps), and persistent data management through file input/output (I/O). A key feature of the application is the robust implementation of explicit input validation and business rule enforcement using control flow, ensuring data integrity without relying on external database software. The application allows users to open accounts, deposit funds withdraw cash, perform inter-account transfers, and generate transaction statements, providing essential balance alerts and financial tracking.

Problem Statement

Manual record-keeping for financial transactions (recording transactions in the ledger by hand) is very ineffective, error-prone and does not allow for current status reports. Accounting mistakes can lead to important errors, such as illegal overdrafts and losses in operations.

Proposed Solution

Banking Management System has a simple interface which can be accessed from the console screen. The system is able to keep a record of transactions and uses the data file for storage operations, as well as making the interface reliable.

Data Structure Design

The banking data is stored within Java as structured object models, leveraging the efficiency of the Java Collections Framework:

Outer Structure (Accounts): A master Map where the Account Number (a string such as "SB-1001" or "CA-2001") acts as the unique key.

Inner Structure (Details): The value associated with the key is an Account object containing encapsulated account details:

- o 'accountHolderName': A string representing customer name.
- o 'balance': A double tracking current monetary balance.
- o 'hashedPin': A string storing salted SHA-256 password hash.
- o 'transactions': A List storing chronological Transaction records.

Goal of this program:-

The principal aim of the initiative is to establish a system for managing accounts for small banking institutions, cooperative branches and clients thus tracking transactions of funds. The use of simple menu navigation and direct access methods of operation makes this a very straightforward system which is equally useful in real life. This approach to account management minimizes human errors and ensures systematic monitoring of activity in accounts to respond to requests promptly.

Explanation of the Program:-

This program begins by organizing domain logic across clean Java classes. The java.io package provides classes to interact with files on the filesystem in a portable way. Here we use object streams to interact with the data files of the operating system to store the state of accounts and access them as and when required by us. We then create several methods that contain the working of how accounts are accessed to get the balances or to store and update the values of transactions in them. The program is mainly built with the help of loops, if-else constructs, and basic console input/output statements to get the values. The program also verifies security PINs before releasing funds to prevent unauthorized access. The application is entirely menu-driven and implements the following essential banking features:

Data Persistence: Utilizes a standard database file (bank_data.dat) for the storage and retrieval of account information as well as recording the account details after the program is run every time.

Account Management: Provides the option of opening accounts (savings accounts made with deposits and interest rates or current accounts made with overdraft) and enables transactions in the form of the deposits and withdrawals of money and Also the tansfer of money between accounts.

Input Validation: Has a unique structure of if else statements that identifies the inputted values correctly by checking for the positive transaction values so as to avoid any logical problems.

Balance Restrictions & Alerts: Automatically checks if the savings account is below the required minimum balace of \$100.00 or if the current account is used in excess of the overdraft limit to prevent a financial catastrophe.

Reporting: Shows account statements and Operation histories in clear tables on the console.

Results:

The main objective of the program, which is to achieve effective control of banking operations through a simple but effective Text user interface that mimics the real-world banking operation, has been achieved.

Conclusion:-

Banking Management System Application is an excellent representation of programming skills in Java. The program makes use of object oriented concepts to organize the data and the modularization using functions. The use of proper if else conditional logic ensures that the validation is done properly. Proper implementation of data persistence proves that the student knows about File I/O and fundamental application design.

What I learnt in this project:-

While making this project on banking management, I learnt the use of the java.io package and how it interacts with the Java program to store values so that they can be accessed anytime when needed. I gained the knowledge of how to store files in the directories of the system and can be used as an information management system also. I also learnt a broader use of if-else and looping constructs. While working out the program, I also got to know about try-catch type of constructs and got to know how it handles exceptions cleanly without stopping the program abruptly, allowing error detection quickly without excessive nested logic statements.

Future Scope

To evolve this project beyond the introductory level, the following enhancements could be implemented:

1. Database Migration: Replace file storage with a relational database (e.g., SQLite or MySQL) for better query performance and structured data management.
2. Graphical User Interface (GUI): Implement a desktop interface using libraries such as Swing or JavaFX to improve user experience.
3. Advanced Alerting: Integrate functions to check and alert the user about low balances or overdraft usage via email or SMS notifications.
4. Reporting: Add the ability to generate printable reports (e.g., PDF or CSV) of all recent transactions or bank liquidity audits.